

CoreOps Week3 Assignments

Theme: Kubernetes Security & Platform Hardening

Objective: Give folks a *practical security posture understanding*.

Everything below is step-by-step and runnable on ANY cluster (minikube, kind, k3s, EKS/GKE).

MONDAY: Pod Security Admission + Seccomp + AppArmor

Lab 1: Enable PSA (Pod Security Admission) in Restricted Mode

Step 1: Create namespace with PSA = restricted

- `kubectl create ns secure-ns`
- `kubectl label ns secure-ns pod-security.kubernetes.io/enforce=restricted`

Step 2: Try deploying a privileged pod (FAIL expected)

`cat <<EOF > bad-pod.yaml`

`apiVersion: v1`

`kind: Pod`

`metadata:`

`name: privileged-pod`

`namespace: secure-ns`

`spec:`

`containers:`

`- name: c1`

`image: busybox`

`command: ["sleep", "3600"]`

`securityContext:`

`privileged: true`

`EOF`

`kubectl apply -f bad-pod.yaml`

Step 3: Debug why it failed

```
kubectl describe pod privileged-pod -n secure-ns
kubectl get events -n secure-ns
```

Expect: **PSA violation**.

Lab 2: Fix the Pod by Removing Insecure Fields

```
cat <<EOF > good-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
  namespace: secure-ns
spec:
  containers:
    - name: c1
      image: busybox
      command: ["sleep", "3600"]
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
EOF
```

- kubectl apply -f good-pod.yaml
- kubectl get pods -n secure-ns

Expect: Pod now runs successfully.

Lab 3: Apply Seccomp Profile

Step 1: Enable seccomp

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: seccomp-pod
  namespace: secure-ns
  annotations:
    container.seccomp.security.alpha.kubernetes.io/c1: runtime/default
spec:
```

```
containers:  
- name: c1  
  image: busybox  
  command: ["sleep", "3600"]  
EOF
```

Step 2: Verify seccomp

```
kubectl get pod seccomp-pod -n secure-ns -o yaml | grep seccomp
```



TUESDAY: RBAC + Namespaces

Lab 1: Create a Namespace for TeamA

```
kubectl create ns team-a
```

Lab 2: Create a Service Account

```
kubectl create sa developer -n team-a
```

Lab 3: Create a Least-Privilege Role

```
cat <<EOF > role.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: team-a
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
EOF
```

- `kubectl apply -f role.yaml`

Lab 4: Bind the SA to the Role

```
kubectl create rolebinding pod-read-binding \
--role=pod-reader \
--serviceaccount=team-a:developer \
-n team-a
```

Lab 5: Validate Access

- `kubectl auth can-i list pods --as=system:serviceaccount:team-a:developer -n team-a`
- `kubectl auth can-i delete pods --as=system:serviceaccount:team-a:developer -n team-a`

Expected: list = yes, delete = no

WEDNESDAY: Service Accounts + Workload Identity

Lab 1: Create SA for an App

- `kubectl create ns identity-lab`
- `kubectl create sa app-sa -n identity-lab`

Lab 2: Deploy Pod Using This SA

`cat <<EOF > sa-pod.yaml`

`apiVersion: v1`

`kind: Pod`

`metadata:`

`name: app-identity`

`namespace: identity-lab`

`spec:`

`serviceAccountName: app-sa`

`containers:`

`- name: c1`

`image: busybox`

`command: ["sh", "-c", "sleep 3600"]`

`EOF`

- `kubectl apply -f sa-pod.yaml`

Lab 3: Verify Token Mounted

`kubectl exec -n identity-lab app-identity -- ls`

`/var/run/secrets/kubernetes.io/serviceaccount`

THURSDAY — Secrets & SealedSecrets

Lab 1: Create Normal Secret

```
kubectl create ns secrets-lab
kubectl create secret generic db-creds \
  --from-literal=username=admin \
  --from-literal=password=pass123 \
  -n secrets-lab
```

Check: `kubectl get secret db-creds -n secrets-lab -o yaml`

Lab 2: Install SealedSecrets Controller

```
kubectl apply -f
https://github.com/bitnami-labs/sealed-secrets/releases/download/v0.24.0/controller.yaml
```

Lab 3: Seal a Secret

```
kubectl create secret generic api-key \
  --from-literal=token=abcd1234 \
  -n secrets-lab \
  --dry-run=client -o yaml > secret.yaml
```

- `kubectl seal < secret.yaml > sealedsecret.yaml`
- `kubectl apply -f sealedsecret.yaml`
- Verify: `kubectl get secrets -n secrets-lab`

FRIDAY: Supply Chain Basics (Cosign Intro + Policy)

Lab 1: Install Cosign

curl -sSL

<https://github.com/sigstore/cosign/releases/download/v2.1.1/cosign-linux-amd64> -o cosign

- `chmod +x cosign`
- `sudo mv cosign /usr/local/bin/`

Lab 2: Generate a Key Pair

`cosign generate-key-pair`

Creates: `cosign.key`, `cosign.pub`

Lab 3: Sign an Example Image

`cosign sign --key cosign.key nginx:latest`

Lab 4: Verify

`cosign verify --key cosign.pub nginx:latest`

THE DEVOPS WAR ROOM